LangGraph-E-M-Set1

20 questions | Easy to medium | Choose one answer per question

01. Which requirement is the best fit for LangGraph?

A. Store document embeddings and rank their similarity.

B. Coordinate a stateful workflow with branches, loops, and human review.

C. Validate incoming API payloads against Python type definitions.

D. Expose Python functions as HTTP endpoints.

02. A report workflow must pass a company name, retrieved evidence, and a draft between steps. What is the role of graph state?

A. It is the Python call stack for the currently running node.

B. It is a prompt template automatically sent to every model.

C. It is an external vector index used by all workers.

D. It is structured data that nodes read and update during execution.

03. A TypedDict-based StateGraph has fields called question, evidence, and answer. A node only changes answer. Which return value is appropriate?

```
A. {"answer": "Approved"}
B. "Approved"
C. {"question": None, "evidence": None, "answer": "Approved"}
D. None, after changing only a local variable named answer
```

04. After adding nodes and edges to a StateGraph builder, what does builder.compile() produce?

A. A vector index containing every node output.

B. A completed run of the graph using default input.

C. An executable graph that can be invoked or streamed.

D. An HTTP server with an endpoint for every node.

LangGraph-E-M-Set1

20 questions | Easy to medium | Choose one answer per question

05. In a simple linear graph, what do the special START and END markers describe?

A. The first and last messages in conversation history.

B. The workflow entry point and a terminal destination.

C. The opening and closing of a database connection.

D. The beginning and end of an individual model response.

06. After evaluation, a workflow should route to revise when quality is low and to publish when quality is acceptable. Which design expresses this choice?

A. Two unconditional edges from evaluate, one to each destination.

B. A reducer on the quality field that automatically chooses a node.

C. A new thread_id for each possible destination.

D. A conditional edge whose routing function selects the destination.

07. What does the add_messages reducer do when it receives a message with the same ID as an existing message?

A. It updates the matching message; messages with new IDs are appended.

B. It always appends a duplicate, even when the ID already exists.

C. It discards the update because message history cannot change.

D. It sorts all messages alphabetically by their IDs.

08. What is the primary job of a LangGraph checkpointer?

A. Evaluate whether an LLM answer is factually correct.

B. Store only the application source code for later deployment.

C. Persist graph execution state so a thread can continue or resume.

D. Automatically convert all node functions into asynchronous functions.

LangGraph-E-M-Set1

20 questions | Easy to medium | Choose one answer per question

09. A compiled graph uses a checkpointer. Which configuration identifies the thread whose state should be saved or loaded?

A. `{"configurable": {"thread_id": "report-42"}}`

B. `{"metadata": {"thread_id": "report-42"}}`

C. `{"configurable": {"run_name": "report-42"}}`

D. `{"tags": ["report-42"]}`

10. Using graph.stream(..., stream_mode="updates"), what kind of information is emitted?

A. Only individual text tokens emitted by an LLM.

B. The entire final state only after the whole graph finishes.

C. Automatically generated distributed tracing dashboards.

D. Node-produced state updates as graph execution progresses.

11. The state schema below is used with operator.add as the reducer for notes. Before a node runs, count is 4 and notes is ["retrieved"]. The node returns {"count": 7, "notes": ["drafted"]}. What is the resulting state?

```
class State(TypedDict):
    count: int
    notes: Annotated[list[str], operator.add]
```

A. `{"count": 11, "notes": ["drafted"]}`

B. `{"count": 7, "notes": ["retrieved", "drafted"]}`

C. `{"count": 11, "notes": ["retrieved", "drafted"]}`

D. `{"count": 7, "notes": ["drafted"]}`

12. Two parallel nodes return updates to the same results field in the same superstep. The field is declared as results: list[str], with no reducer. What should you expect?

A. LangGraph always concatenates the two lists automatically.

B. LangGraph silently keeps the output of the slower node.

C. A concurrent-update error because the field cannot merge both writes.

D. LangGraph automatically serializes the two nodes by name.

LangGraph-E-M-Set1

20 questions | Easy to medium | Choose one answer per question

13. An orchestrator discovers a different number of report sections for each request. Each section needs its own worker input. Which feature directly supports this dynamic fan-out?

A. Return Send("write_section", section_input) objects from routing logic.

B. Add one static edge to write_section and let it automatically fan out the list items.

C. Attach a list reducer to sections and let the reducer schedule a worker per item.

D. Create a thread_id for each section and expect the parent graph to join them automatically.

14. Two branches must both finish before assemble runs. Which edge declaration explicitly makes assemble wait for both section_a and section_b?

A. `builder.add_edge("section_a", "assemble")`

B. `builder.add_edge("section_b", "assemble")`

C. `builder.add_edge("assemble", END)`

D. `builder.add_edge(["section_a", "section_b"], "assemble")`

15. A node needs to set approved to True and route to publish in one return value. Assume it has no static outgoing edges. Which return value directly expresses both operations?

A. {"approved": True}

B. Command(update={"approved": True}, goto="publish")

C. Send("publish", {"approved": True}) returned directly as a normal state update

D. Command(resume=True)

16. A graph with a checkpointer pauses at approved = interrupt("Approve report?"). Which call supplies True as the review decision and resumes that paused execution?

A. `graph.invoke({"approved": True}, config=new_thread_config)`

B. `graph.invoke(None, config=new_thread_config)`

C. `graph.invoke(Command(resume=True), config=original_thread_config)`

D. `graph.invoke(Command(goto="publish"), config=original_thread_config)`

LangGraph-E-M-Set1

20 questions | Easy to medium | Choose one answer per question

17. A node calls create_ticket() and then interrupt() for approval. create_ticket() creates a new external ticket on every call. When the interrupted node resumes, what risk must the developer handle?

A. The code before interrupt() can run again, creating a duplicate ticket.

B. LangGraph automatically deletes the original ticket before continuing.

C. Resuming permanently disables every later call to create_ticket().

D. A checkpointer guarantees the external ticket is created exactly once.

18. A locally hosted graph uses InMemorySaver. It must support resuming paused reports after future Python process restarts. Which design change is required?

A. Increase recursion_limit while keeping the in-memory checkpointer.

B. Use stream_mode="updates" to retain state after shutdown.

C. Reuse the same thread_id without changing the storage backend.

D. Use a persistent checkpointer and resume with the same thread identity.

19. A draft-evaluate-revise cycle may keep finding problems. Which design best bounds execution while preserving a useful outcome?

A. Route every evaluation failure straight back to revise with no counter.

B. Track revision attempts, exit or escalate at a limit, and retain a recursion-limit safeguard.

C. Increase recursion_limit whenever the graph reaches it, without changing routing.

D. Route all outputs to publish regardless of the evaluator result.

20. A user starts separate report threads but wants the same writing preferences available in all of them. Which design best matches LangGraph's persistence model?

A. Assume each new thread automatically inherits every other thread's checkpoints.

B. Use one global thread_id for all users and all reports.

C. Store preferences in a Store under a user-scoped namespace and load them when needed.

D. Keep preferences only in the local variables of the first thread's node.